

Backend платформа Go

Цель интервью

Проверяем общие знания кандидата по конкретному языку, устройству компьютера и работе сетей в глубину.

До интервью

- [Создаем](#) новый отчет;
- Заполняем поля «Кандидат», «Интервьюер(ы)» и «Дата».

Как начать интервью

Привет, меня зовут %ИМЯ%, я работаю в Авито в %отделе подсчета денег%. Ничего если я буду на ты? Сегодня у нас с тобой секция платформы. На ней мы должны обсудим широкий спектр вопросов связанных с языком программирования Go.

Сейчас я скину ссылку на блокнотик [avito.code](#) где мы будем писать код.

Как проводить интервью

- Интервью с кандидатом начинаем с решения любой задачи из списка ниже (*Почему нужны задачи? Потому что типичная история, когда человек “выучил” теорию, но на практике применить не может.*);
- Во время решения задачи строим диалог о понимании кандидатом соответствующих технологий, стараясь покрыть как базовые особенности языка (указатели, горутины, каналы, средства синхронизации и работа с контекстом), так и особенности работы с сетью;
- Артефактом решения задачи должен стать полный текст задачи в файле отчета и решение предложенное кандидатом, а также ответы кандидата на дополнительные вопросы;
- На случай, если осталось время пообщаться или какой-то аспект не покрыт практикой - можно взять пару дополнительных вопросов из секции [Теоретические вопросы](#).

Как закончить интервью

У меня вопросов больше нет. Может ты хочешь что-нибудь спросить у меня? Если нет - я могу рассказать как тут всё устроено в Авито или что будет дальше на следующих этапах.

%отвечаем на вопросы, объясняем остальные секции и примерные сроки; попутно рассказываем, за что вы сами любите Авито - помогает продать вакансию%

Спасибо за то что пришел(ла), было приятно с тобой пообщаться, удачи на следующих этапах.

После интервью

- «Причесываем» заметки по ответам кандидата;
- Заполняем поля «Ссылка на код», «Оценка уровня» и оставляем комментарий на основе чего было принято это решение;
- Нажимаем «Сохранить отчет»;
- Переходим на детальную страницу интервью по ссылке в календаре (приглашении), добавляем ссылку на отчет в поле «Ссылка на итоги», выставляем «Уровень» и завершаем интервью.

Инструменты

- Для ведения [протокола](#)
- Для онлайн-кодинга с [подсветкой](#) или [исполнением](#)
- Онлайн whiteboard: [Draw.io](#), [Miro](#), [Whimsical](#) или [Excalidraw](#)

Рекомендуемые задачи

[Указатели \(easy\)](#)

Условие задачи

Что выведет следующая программа и почему?

```
package main

import "fmt"

type Person struct {
    Name string
}

func changeName(person *Person) {
    person = &Person{
        Name: "Alice",
    }
}
```

```
func main() {  
    person := &Person{  
        Name: "Bob",  
    }  
    fmt.Println(person.Name)  
    changeName(person)  
    fmt.Println(person.Name)  
}
```

Ответ:

```
Bob  
Bob
```

Как модифицировать программу, чтобы вывелось:

```
Bob  
Alice
```

Что проверяем

- Базовое понимание структур
- Понимание работы с указателями

Примечание

- Функция `changeName` принимает копию указателя и в копию указателя записывается адрес нового объекта.
- [There is no pass-by-reference in Go](#)

Горутины в цикле (easy)

Условие задачи

2 варианта вопроса:

- Что выведет следующая программа и почему?
- Можно попросить кандидата "провести code-review коллеги и найти проблемы, если есть". Саму логику того, что это через горутины не меняем.

```
// Находим максимальное четное число
```

```
package main
```

```
import "fmt"

func main() {
    var max int

    for i := 1000; i > 0; i-- {
        go func() {
            if i%2 == 0 && i > max {
                max = i
            }
        }()
    }

    fmt.Printf("Maximum is %d", max)
}
```

Что проверяем

- Базовое знание горутин
- Понимание областей видимости переменных
- Знание примитивов синхронизации
- Понимание проблем race conditions

Примечание

Проблема 1: Замыкание

- Вывод не соответствует тому, что мы ожидаем - так как замыкание `i` в цикле связывает значение переменной (и на это ругнётся `go vet` при сборке - `./prog.go:10:25: loop variable i captured by func literal`). Каким будет `i` в момент запуска первой горутины мы не знаем.
- Решение: передать `i` как параметр в анонимную функцию

Проблема 2: Синхронизация

- Программа завершается не обязательно дождавшись присвоения хотя бы одного числа (запуска хотя бы одной горутины).
- Решение: синхронизировать ожидание через `sync.WaitGroup`
- Дополнительный вопрос к секции: если кандидат использует `defer` для `wg.Done()` спросить осознанность его решения и почему он не написал

линейно без defer. Можно уточнить тут про потенциальную доп. нагрузку, которую дает (или не дает defer). Поговорить про Low-cost defers

Проблема 3: race conditions

- Переменная max будет использоваться множеством горутин, вызывая race condition. Значение переменной в итоге может получиться любым
- Решение: можно решить разными путями, но базово - через Mutex. Важная ремарка - если править через Атомики только строчку присвоения - проблема не решится. Лочить обязательно необходимо также и строку со сравнением.
- Дополнительно: тут важно понять насколько кандидат вообще понимает принципы concurrency в ходе его рассуждений о решении. Если он сразу поставит лок перед условием - спросить "зачем мы лочим вообще все, если читать можно конкурентно без проблем"

Дополнительные вопросы:

- Какие есть инструменты (и есть ли) в go, которые могут помочь найти проблемы выше (go vet, -race флаг и тд)

Сборка сниппета (easy)

Условие задачи

Мы делаем сервис, который собирает сниппет товара. Каждый сниппет состоит из отформатированного описания и стоимости товара в рублях.

Чтобы собрать сниппет нужно:

- Получить описание из одного сервиса, а затем отформатированное его через `prettify`
- Получить цену (в долларах) из другого сервиса, а затем перевести ее в рубли через `priceToRub`
- Вернуть готовый сниппет

Дополнительные вопросы

- Как распараллелить пачку задач и дождаться завершения всех?
 - Работа с общей памятью, атомарность операций и необходимость синхронизации

- Предположим, что сетевые запросы теперь могут выдать ошибку, как это обработать?
 - Какие есть варианты? Показать цену в долларах или спятисотить или ретраить?
- Какой таймаут правильно выставить на вызовы во внешние сервисы?
- Асинхронность:
 - Отличие между тредами, процессами и корутинами?
 - Как работает веб-сервер?
 - Как устроен шедюлер, как устроены приоритеты, в какие моменты корутины засыпают и просыпаются?
- Синхронизация:
 - Мьютексы и как ими пользоваться (лок на чтение и на запись, отпускание локов после exception'ов)?
 - Атомарные операции, compare-and-swap etc, rw-мьютексы, спинлоки, как оно работает внутри?
 - Синхронизация между процессами на одной машине и между разными машинами, типовые способы ее избежать?

Заготовка кода

[Go](#)

[Python](#)

[C++](#)

[C#](#)

[Java](#)

[NodeJS](#)

Пример решения

[Go](#)

```
package main
```

```
func BuildSnippet(itemId int) Snippet {  
    wg := sync.WaitGroup{}  
    wg.Add(2)
```

```

var desc string
go func() {
    rawDesc := itemDescription(itemId)
    desc = prettify(rawDesc)
    wg.Done()
}()

var priceRub float64
go func() {
    priceDollars := itemPrice(itemId)
    priceRub = priceToRub(priceDollars)
    wg.Done()
}()

wg.Wait()

return Snippet{
    Price:    priceRub,
    Description: desc,
}
}

```

[Python](#)

[C++](#)

[C#](#)

[Java](#)

[NodeJS](#)

[Прогноз погоды \(medium\)](#)

Условие задачи

- Есть функция которая через нейронную сеть вычисляет прогноз погоды за ~1 секунду
- Есть highload RPC ручка с нагрузкой 10k RPS
- Необходимо реализовать код этой ручки

Дополнительные вопросы

- А что если нам теперь приходит параметр `cityId` и нам надо отдавать погоду для него?
- Что будет если каждый раз ходить в микросервис?
 - каждый запрос будет обрабатываться по секунде, а это дополнительный расход памяти, CPU и т.д
- Какой таймаут правильно выставить на ручку вызов `aiWeatherForecast`?
- Если человек додумался сделать кеш со словариком, то какие нештатные ситуации могут случиться?
 - Прогрев кеша, протухание кеша, одновременное протухание кеша и т.п..
 - Как предотвратить эти ситуации?
- Как выглядит HTTP-запрос и ответ
 - Method, URL, Headers, Body
 - Статусы HTTP-ответов (нужно принципиально отличать 200, 400 и 500, знать как их обрабатывать)
 - Путь запроса от пользовательского приложения до сервера и обратно (шифрование, сокет, буферы, DNS, IP, роутеры, MTU, таблицы маршрутизации, фаерволы, почему витая пара витая)

Заготовки кода

[Go](#)

```
package main
```

```
// aiWeatherForecast через нейронную сеть вычисляет прогноз погоды за ~1 секунду
```

```
func aiWeatherForecast() int {
    time.Sleep(1 * time.Second)
    return rand.Intn(70) - 30
}
```

```
func main() {
    http.HandleFunc("/weather", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "{\"temperature\":%d}\n", aiWeatherForecast())
    })
    if err := http.ListenAndServe(":3333", nil); err != nil {
        panic(err)
    }
}
```

Экспертные вопросы

- Какую структуру правильно выбрать для кэша?
- Как исключить гонки при паралельном доступе к структуре?
- Как инвалидировать кеш?
- Как реализовать прогрев кеша перед стартом приложения?

[Python](#)

```
import random
import time
import json

def ai_weather_forecast():
    time.sleep(1)
    return random.randint(-30, 39)

@app.route('/weather', methods=['GET'])
def get_weather():
    return json.dump({'temperature': ai_weather_forecast()})
```

Экспертные вопросы

- Какую структуру правильно выбрать для кэша?
- Как исключить гонки при паралельном доступе к структуре?
- Как инвалидировать кеш?
- Как реализовать прогрев кеша перед стартом приложения?

[C++](#)

```
#include <cpprest/http_listener.h>
#include <cpprest/filestream.h>
#include <random>
#include <chrono>

using namespace utility;
using namespace web;
using namespace web::http;
using namespace web::http::experimental::listener;

// aiWeatherForecast через нейронную сеть вычисляет прогноз погоды за ~1 секунду
int aiWeatherForecast() {
    std::this_thread::sleep_for(std::chrono::seconds(1));
    std::default_random_engine generator;
    std::uniform_int_distribution<int> distribution(-10, 36);
    return distribution(generator);
}
```

```

}

void handle_get(http_request request) {
    auto forecast = aiWeatherForecast();
    request.reply(http_status_codes::OK, u8"{" + utility::to_string(forecast) + "}");
}

int main() {
    try {
        http_listener listener(U("http://localhost:3333/weather"));
        listener.support(methods::GET, handle_get);
        listener.open().wait();

        std::cout << U("Listening on http://localhost:3333/weather\n");
        while (true) {
            std::this_thread::sleep_for(std::chrono::milliseconds(1000));
        }
    } catch (const std::exception &e) {
        std::cerr << e.what() << std::endl;
    }

    return 0;
}

```

Экспертные вопросы

- Какую структуру правильно выбрать для кэша?
 - Оптимально unordered_map, просто map тоже можно, отдельно можно поговорить "что тут лучше"
 - Если предложит вектор, то рассказать, что city_id это не плотный диапазон, а скажем внешний хэшкод
- Как исключить гонки при паралельном доступе к структуре кэша?
 - Канонично rwlock, можно mutex, но тогда выяснить "знаешь ли про rwlock"?
 - Копнуть в понимание (или нет), что вставка в mapу под wlock может приводить к небольшим паузам
 - На оценку 5+ понимание, что под high contention + fast requests mutex это лучше rwlock, но 10 krps скорее всего not high enough

C#

```
namespace Weather.Controllers;
```

```

[ApiController]
[Route("[controller]")]
public class WeatherForecastController : ControllerBase
{
    private readonly IWeatherForecastService _weather;
    public WeatherForecastController(IWeatherForecastService weather)
    {
        _weather = weather;
    }

    [HttpGet(Name = "weather")]
    public async Task<int> Get()
    {
        return await _weather.Get();
    }
}

public interface IWeatherForecastService
{
    Task<int> Get();
}

class WeatherForecastService : IWeatherForecastService
{
    private readonly Random _rnd;

    public WeatherForecastService()
    {
        _rnd = new Random();
    }

    public async Task<int> Get()
    {
        await Task.Delay(1 * 1000);
        return _rnd.Next(70) - 30;
    }
}

```

Экспертные вопросы

- Какую структуру правильно выбрать для кэша?
 - Что лучше для InMemory кэша mutex+map или ConcurrentDictionary?
- Как исключить гонки при параллельном доступе к структуре?
 - Какие примитивы синхронизации есть?

- Как инвалидировать кеш?
- Как реализовать прогрев кеша перед стартом приложения?
- Как запустить background service?

[Java](#)

```
public class MyWebServer {
    // aiWeatherForecast через нейронную сеть вычисляет прогноз погоды за ~1 секунду
    private int aiWeatherForecast() {
        Thread.sleep(1000);
        return new Random().nextInt(70) - 30;
    }

    static class WeatherHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            int temperature = aiWeatherForecast();
            String jsonResponse = "{\"temperature\": " + temperature + "}";

            OutputStream os = exchange.getResponseBody();
            os.write(jsonResponse.getBytes());
            os.close();
        }
    }
}
```

Экспертные вопросы

- Структура для кеша должна либо гарантировать thread safe операции, либо реализация решения должна включать эти гарантии с помощью synchronized операции либо других решений из пакета java.util.concurrent
- Прогрев кеша перед стартом приложения / актуализация для top N запросов
- Инвалидация кеша по TTL или другому принципу
- Реализация параллелизма, простой Thread, CompletableFuture, возможные эффекты
- Обработка ошибок и возвращаемый response клиенту
- Как реализовать лимиты по таймаутам для запроса
- Как реализовать кеш для N сервисов

[NodeJS](#)

```
// aiWeatherForecast через нейронную сеть вычисляет прогноз погоды за ~1 секунду
```

```

function aiWeatherForecast() {
    return Math.floor(Math.random() * 70) - 30
}

async function main() {
    http.HandleFunc("/weather", async function (resp, req) {
        res.end(JSON.stringify({
            "temperature": aiWeatherForecast()
        }));
    })
}

```

Экспертные вопросы

- Какую структуру правильно выбрать для кэша?
- Как исключить гонки при параллельном доступе к структуре?
- Как инвалидировать кеш?
- Как реализовать прогрев кеша перед стартом приложения?

Пример решения

[Go без локаций](#)

```

package main

func aiWeatherForecast() int {
    time.Sleep(1 * time.Second)
    return rand.Intn(70) - 30
}

var temperature int
var mu = sync.RWMutex{}

func updateWeather() {
    value := aiWeatherForecast()
    mu.Lock()
    temperature = value
    mu.Unlock()
}

func main() {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

```

```
updateWeather()
```

```
closed := make(chan interface{})
```

```
go func(ctx context.Context) {
```

```
    defer close(closed)
```

```
    for {
```

```
        select {
```

```
        case <-ctx.Done():
```

```
            return
```

```
        case <-time.After(1 * time.Second):
```

```
        }
```

```
        updateWeather()
```

```
    }
```

```
}(ctx)
```

```
http.HandleFunc("/weather", func(w http.ResponseWriter, r *http.Request) {
```

```
    mu.RLock()
```

```
    fmt.Fprintf(w, "{\"temperature\":%d}\n", temperature)
```

```
    mu.RUnlock()
```

```
})
```

```
if err := http.ListenAndServe(":3333", nil); err != nil {
```

```
    panic(err)
```

```
}
```

```
<-closed
```

```
}
```

[Go с локациями](#)

[C++ без локаций](#)

[C++ с локациями](#)

```
// aiWeatherForecast через нейронную сеть вычисляет прогноз погоды за ~1 секунду
```

```
int aiWeatherForecast() {
```

```
    mySleep(1000);
```

```
    return myRandom();
```

```
}
```

```
struct ParsedLocationRequest {
```

```
    int cityId;
```

```
};
```

```

struct HttpResponse {
    int status;
    std::string body;
};

class MyWebServer
{
public:
    void execute() {
        while(true) {
            swapStorage();
            sleep(timer);
            catch_signal();
        }
    }
private:
    void swapStorage() {
        std::unique_lock lock(storageMtx);
        for(const auto& [cityId, weather] : storage_) {
            oldStorage_[cityId] = weather;
        }
        storage_.clear();
    }

    struct Entry {
        int weather{};
        timestamp reqTime;
    }

    std::unordered_map<cityId, Entry> storage_;
    std::unordered_map<cityId, Entry> oldStorage_;

    timer = 1m;
    std::mutex storageMtx_;
};

void MyWebServer::updateStorage(int cityId, time reqTime) {
    auto newWeather = aiWeatherForecast(cityId);
    auto& newEntry = storage[cityId];
    newEntry.weather = newWeather;
    newEntry.reqTime = reqTime;
    return HttpResponse("200", weather);
}

```

```

}

// 10 krps
HttpResponse MyWebServer::Handle(ParsedLocationRequest req)
{
    const auto& cityId = req.cityId;
    const auto& reqTime = req.timestamp;

    std::shared_lock lock(storageMtx_);
    auto it = oldStorage_.find(cityId);
    if(it != oldStorage_.end()) {
        if((reqTime - it.second.reqTime > 1h)) {
            return updateStorage(cityId, it.second.reqTime);
        } else {
            return HttpResponse("200", it->second.weather);
        }
    }
    return updateStorage(cityId, reqTime);
}

```

[C# без локаций](#)

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Caching.Memory;

namespace Weather.Controllers;

[ApiController]
[Route("[controller]")]
public class SimpleWeatherForecastController : ControllerBase
{
    private readonly IWeatherForecastService _weather;

    public SimpleWeatherForecastController(IWeatherForecastService weather)
    {
        _weather = weather;
    }

    [HttpGet(Name = "weather")]
    public async Task<int> Get(Cancellation token = default)
    {
        return await _weather.Get(token);
    }
}

```



```
public interface IWeatherForecastService
{
    Task<int> Get(Cancellation token = default);
}
```

```
public interface IWeatherForecastUpdater
{
    Task StartUpdate();
    void StopUpdate();
}
```

```
public class SimpleWeatherService : IWeatherForecastService, IWeatherForecastUpdater
{
    private readonly SemaphoreSlim _semaphoreSlim;
    private readonly IMemoryCache _cache;
    private readonly Random _rnd;
    private const int DefaultWeatherId = 0;
    private const int CalcTimeoutSeconds = 1;
    private const int CacheSeconds = 60;
    private CancellationSource _cancellationSource;

    public SimpleWeatherUpdater(IMemoryCache cache)
    {
        _rnd = new Random();
        _cache = cache;
        _cancellationSource = new CancellationSource();
        _semaphoreSlim = new SemaphoreSlim(1, 1);
    }

    public async Task<int> Get(Cancellation token = default)
    {
        if (_cache.TryGetValue(DefaultWeatherId, out int w))
            return w;

        return await ExecSingle(token);
    }

    /// <summary>
    /// Остановить обновление справочника
    /// </summary>
    public void StopUpdate()
    {
        _cancellationSource.Cancel();
    }
}
```

```

    }

    /// <summary>
    /// Начать обновление справочника
    /// </summary>
    public async Task StartUpdate()
    {
        _cancellationSource = new CancellationTokenSource();
        using var tickTimer = new PeriodicTimer(TimeSpan.FromSeconds(CalcTimeoutSeconds));

        while (await tickTimer.WaitForNextTickAsync(_cancellationSource.Token))
        {
            await ExecSingle(_cancellationSource.Token);
        }
    }

    private async Task<int> ExecSingle(CancellationToken token)
    {
        if (token.IsCancellationRequested)
            throw new OperationCanceledException();

        try
        {
            await _semaphoreSlim.WaitAsync(token);

            if (_cache.TryGetValue(DefaultWeatherId, out int w))
                return w;

            await Task.Delay(CalcTimeoutSeconds * 1000, token);
            w = _rnd.Next(70) - 30;

            _cache.Set(DefaultWeatherId, w,
                new
                MemoryCacheEntryOptions().SetAbsoluteExpiration(TimeSpan.FromSeconds(CacheSeconds)));

            return w;
        }
        finally
        {
            _semaphoreSlim.Release();
        }
    }
}

```

[C# с локациями](#)

```
using System.Collections.Concurrent;
using Microsoft.AspNetCore.Mvc;

namespace Weather.Controllers;

[ApiController]
[Route("[controller]")]
public class WeatherForecastController : ControllerBase
{
    private readonly IWeatherForecastService _weather;

    public WeatherForecastController(IWeatherForecastService weather)
    {
        _weather = weather;
    }

    /// <summary>
    /// Получение прогноза погоды
    /// </summary>
    /// <param name="locationId"></param>
    /// <param name="token"></param>
    /// <returns></returns>
    [HttpGet(Name = "GetWeather")]
    public async Task<int> Get(int? locationId, CancellationToken token = default)
    {
        return await _weather.Get(locationId, token);
    }
}

public interface IWeatherForecastService
{
    Task<int> Get(int? locationId, CancellationToken token = default);
}

public interface IWeatherUpdater
{
    Task StartUpdate();
    void StopUpdate();
}

public class WeatherService : IWeatherForecastService, IWeatherUpdater
{

```

```

private readonly IMemoryCache _cache;
private readonly List<int> _requiredLocations;
private readonly ConcurrentDictionary<int, SemaphoreSlim> _semaphores;
private readonly Random _rnd;
private const int DefaultWeatherId = 0;
private const int CalcTimeoutSeconds = 60;
private const int CacheSeconds = 60;
private CancellationTokenSource _cancellationSource;

```

```

public WeatherUpdater(IMemoryCache cache)
{
    _cache = cache;
    _rnd = new Random();
    _cancellationSource = new CancellationTokenSource();
    _requiredLocations = new List<int>();
    _semaphores = new ConcurrentDictionary<int, SemaphoreSlim>();
}

```

```

/// <summary>
/// Ручка получения прогноза погоды
/// </summary>
/// <param name="locationId">id локации</param>
/// <param name="token">токен отмены</param>
/// <returns>Температура</returns>
public async Task<int> Get(int? locationId, CancellationToken token = default)
{
    var key = GetWeatherKey(locationId);
    if (_cache.TryGetValue(key, out var weather))
    {
        return weather;
    }

    return await ExecSingle(key, token);
}

```

```

/// <summary>
/// Остановить обновление справочника
/// </summary>
public void StopUpdate()
{
    _cancellationSource.Cancel();
}

```

```

/// <summary>

```

```
/// Начать обновление справочника
```

```
/// </summary>
```

```
public async Task StartUpdate()
```

```
{
```

```
    _cancellationSource = new CancellationTokenSource();
```

```
    using var tickTimer = new PeriodicTimer(TimeSpan.FromSeconds(CalcTimeoutSeconds));
```

```
    // здесь наполняем _requiredLocations значениями locationId
```

```
    // например, из конфига (надо дописать код для этого)
```

```
    while (await tickTimer.WaitForNextTickAsync(_cancellationSource.Token))
```

```
    {
```

```
        await Exec(_cancellationSource.Token);
```

```
    }
```

```
}
```

```
private async Task Exec(CancellationToken token)
```

```
{
```

```
    if (token.IsCancellationRequested)
```

```
        throw new OperationCanceledException();
```

```
    var tasks = new List<Task>();
```

```
    foreach (var locationId in _requiredLocations)
```

```
    {
```

```
        var key = GetWeatherKey(locationId);
```

```
        tasks.Add(ExecSingle(key, token));
```

```
    }
```

```
    await Task.WhenAll(tasks).WaitAsync(token);
```

```
}
```

```
private async Task<int> ExecSingle(int key, CancellationToken token)
```

```
{
```

```
    if (token.IsCancellationRequested)
```

```
        throw new OperationCanceledException();
```

```
    semaphore = _semaphores.GetOrAdd(key, new SemaphoreSlim(1, 1))
```

```
    try
```

```
    {
```

```
        await semaphore.WaitAsync(token);
```

```
        if (_cache.TryGetValue(key, out int w))
```

```
            return w;
```

```

        await Task.Delay(CalcTimeoutSeconds * 1000, token);

        w = _rnd.Next(70) - 30;
        _cache.Set(key, w,
            new
MemoryCacheEntryOptions().SetAbsoluteExpiration(TimeSpan.FromSeconds(CacheSeconds)));

        return w;
    }
    finally
    {
        semaphore.Release();
    }
}

private static int GetWeatherKey(int? key)
{
    return key ?? DefaultWeatherId;
}
}

```

[Java без локаций](#)

```

import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.util.Random;

public class MyWebServerSimple {
    private static final int serverPort = 8000;
    private static final int weatherTtlMs = 1000;

    public static void main(String[] args) throws Exception{
        HttpServer server = HttpServer.create(new InetSocketAddress(serverPort), 0);
        server.createContext("/weather", new WeatherHandler(weatherTtlMs));
        server.setExecutor(null);
        server.start();
    }
}

```

```

class WeatherHandler implements HttpHandler {

    private final int weatherTtl;

    // use AtomicInteger
    private int temperature;

    public WeatherHandler(int weatherTtl) throws InterruptedException {
        this.weatherTtl = weatherTtl;
        this.temperature = aiWeatherForecast();

        updateWeatherByTtl();
    }

    private void updateWeatherByTtl() {
        // use CompletableFuture
        new Thread() -> {
            // case when need to stop
            while (true) {
                try {
                    Thread.sleep(weatherTtl);
                    temperature = aiWeatherForecast();
                } catch (InterruptedException e) {
                    throw new RuntimeException(e);
                }
            }
        }.start();
    }

    @Override
    public void handle(HttpExchange exchange) throws IOException {
        String jsonResponse = "{\"temperature\": " + temperature + "}";

        // handle non success cases and return error http code and response
        exchange.sendResponseHeaders(200, jsonResponse.length());
        OutputStream os = exchange.getResponseBody();
        os.write(jsonResponse.getBytes());
        os.close();
    }

    // aiWeatherForecast через нейронную сеть вычисляет прогноз погоды за ~1 секунду
    private int aiWeatherForecast() throws InterruptedException {
        Thread.sleep(1000);
        return new Random().nextInt(70) - 30;
    }
}

```

```
}  
}
```

Java с локациями

```
import com.sun.net.httpserver.HttpExchange;  
import com.sun.net.httpserver.HttpHandler;  
import com.sun.net.httpserver.HttpServer;
```

```
import java.io.IOException;  
import java.io.OutputStream;  
import java.net.InetSocketAddress;  
import java.util.*;  
import java.util.concurrent.CompletableFuture;  
import java.util.regex.Matcher;  
import java.util.regex.Pattern;  
import java.util.stream.Collectors;  
import java.util.stream.Stream;
```

```
// you can run this example
```

```
public class MyWebServerAdvanced {  
    private static final int serverPort = 8000;  
    private static final int topNLocations = 1000;  
    private static final int weatherTtlMs = 1000;  
  
    public static void main(String[] args) throws Exception{  
        HttpServer server = HttpServer.create(new InetSocketAddress(serverPort), 0);  
        server.createContext("/weather", new WeatherHandlerAdvanced(weatherTtlMs,  
topNLocations));  
        server.setExecutor(null);  
        server.start();  
    }  
}
```

```
class WeatherHandlerAdvanced implements HttpHandler {  
  
    private final Pattern locationIdPattern = Pattern.compile("locationId=\\d*");  
    private final int weatherCacheRefreshIntervalMs;  
    private final int topNLocations;  
    private final Map<Integer, Integer> weatherForecastCache;  
  
    public WeatherHandlerAdvanced(int weatherCacheRefreshIntervalMs, int topNLocations) {  
        this.weatherCacheRefreshIntervalMs = weatherCacheRefreshIntervalMs;
```



```
this.topNLocations = topNLocations;  
// use concurrent implementation  
this.weatherForecastCache = new HashMap<>(topNLocations);
```

```
// implement scheduling for update weather of topN locations by defined ttl  
warmUpWeatherForecastCache();  
}
```

@Override

```
public void handle(HttpExchange exchange) throws IOException {  
    Integer locationId = parseLocationId(exchange);
```

```
    if (locationId == null) {  
        sendResponse(  
            exchange,  
            400,  
            "{\"error\": \"Location id is not defined\"}"  
        );  
        return;  
    }
```

```
    try {  
        sendResponse(  
            exchange,  
            200,  
            "{\"temperature\": " + getTemperature(locationId) + "}"  
        );  
    } catch (InterruptedException e) {  
        sendResponse(  
            exchange,  
            500,  
            "{\"error\": \"Receive actual weather is fault\"}"  
        );  
    }  
}
```

```
private static void sendResponse(HttpExchange exchange, int responseCode, String  
jsonResponse) throws IOException {  
    exchange.sendResponseHeaders(responseCode, jsonResponse.length());  
    OutputStream os = exchange.getResponseBody();  
    os.write(jsonResponse.getBytes());  
    os.close();  
}
```

```

// emulate getting top locations by weather requests
private Set<Integer> getTopNLocations(Integer topNLocations) {
    return Stream.generate(() -> new Random().nextInt(100_000))
        .distinct()
        .limit(topNLocations)
        .collect(Collectors.toSet());
}

private Integer getTemperature(Integer locationId) throws InterruptedException {
    Integer temperature = weatherForecastCache.get(locationId);

    if (temperature != null) {
        return temperature;
    }

    Integer actualTemperature = aiWeatherForecast(locationId);

    // use specific non-common thread pool
    CompletableFuture.runAsync(() -> {
        weatherForecastCache.put(locationId, actualTemperature);
    });

    return actualTemperature;
}

// may produce exception, need to be handled
private Integer parseLocationId(HttpExchange exchange) {
    Matcher matcher = locationIdPattern.matcher(exchange.getRequestURI().getQuery());

    if (!matcher.find()) {
        return null;
    }

    return Integer.parseInt(matcher.group(0).split("=")[1]);
}

// aiWeatherForecast через нейронную сеть вычисляет прогноз погоды за ~1 секунду
private int aiWeatherForecast(Integer locationId) throws InterruptedException {
    Thread.sleep(1000);
    return new Random().nextInt(70) - 30;
}

private void warmUpWeatherForecastCache() {
    for (Integer locationId : getTopNLocations(topNLocations)) {
        // use specific non-common thread pool

```

```

    CompletableFuture.runAsync() -> {
      try {
        Optional.of(aiWeatherForecast(locationId)).ifPresent(temperature ->
weatherForecastCache.put(locationId, temperature));
      } catch (InterruptedException e) {
        // handle exception
        throw new RuntimeException(e);
      }
    });
  };
}
}

```

[NodeJS без локаций](#)

```

function aiWeatherForecast() {
  return Math.floor(Math.random() * 70) - 30
}

```

```

const TTL = 1000;

```

```

async function main() {
  let temperature = aiWeatherForecast();

```

```

  setInterval(() => {
    temperature = aiWeatherForecast();
  }, TTL);

```

```

  http.HandleFunc("/weather", async function (resp, req) {
    res.end(JSON.stringify({
      "temperature": temperature
    }));
  })
}

```

[NodeJS с локациями](#)

```

function aiWeatherForecast(cityId) {
  return Math.floor(Math.random() * 70) - 30
}

```

```

const cache = {};
const TTL = 1000;

```

```

async function main() {
  http.HandleFunction( '/weather',
    async function (req = { cityId: 0 }, resp) {
      const { cityId } = req;
      if (cache[cityId] && cache[cityId].validUntil >= Date.now()) {
        return cache[cityId].value;
      }
      const newValue = await aiWeatherForecast(cityId);
      cache[cityId] = {
        value: newValue,
        validUntil: Date.now() + TTL
      };
      return newValue;
    }
  )
}

```

Сетевые запросы (easy)

Условие задачи

Что выведет следующая программа и сколько она будет выполняться по времени??

```

package main

import (
    "fmt"
    "time"
)

const numRequests = 10000

var count int

func networkRequest() {
    time.Sleep(time.Millisecond) // Эмуляция сетевого запроса.
    count++
}

func main() {
    for i := 0; i < numRequests; i++ {
        networkRequest()
    }
}

```

```
fmt.Println(count)
}
```

Ответ: 10000 и около 10 секунд

Как ускорить выполнение этой задачи?

Что проверяем

- Общее понимание CPU и IO-bound задач
- Базовое знание горутин
- Знание примитивов синхронизации

Примечание

- Кандидат добавляет `go - go networkRequest()`
- Кандидат добавляет `sync.WaitGroup`
- Кандидат защищает `count` локом из пакета `sync` (или использовать пакет `sync/atomic`)
- Какие преимущества работы с `atomic` над `mutex` (рассказывает про специальные инструкции процессоров и преимущества над мютексами)
- Спрашиваем за сколько по его мнению отработает программа теперь (1-2ms норм ответ)
- А что если это настоящий сетевой запрос, не столкнемся ли мы с какой-нибудь проблемой на практике? (лимит файловых дескрипторов)

Параллельный запрос URL адресов (easy)

Условие задачи

- Написать код, который будет выводить коды ответов на HTTP-запросы по двум URL адресам (например главная страница Google и главная страница Avito)
- Запросы должны осуществляться параллельно.

Что проверяем

- Базовое знание горутин.
- Знание примитивов синхронизации
- Понимание работы `context`
- Паттерны межсервисного взаимодействия

Пример решения

```
package main

import (
    "fmt"
    "net/http"
    "sync"
)

func main() {
    var wg sync.WaitGroup
    wg.Add(2)
    go func() {
        defer wg.Done()
        resp, err := http.Get("https://google.com")
        if err != nil {
            fmt.Println(err)
            return
        }
        fmt.Printf("Google status: %s\n", resp.Status)
    }()

    go func() {
        defer wg.Done()
        resp, err := http.Get("https://avito.ru")
        if err != nil {
            fmt.Println(err)
            return
        }
        fmt.Printf("Avito status: %s\n", resp.Status)
    }()

    wg.Wait()
}
```

Возможные ошибки

- Запросы происходят не параллельно
- При использовании WaitGroup вызов wg.Add() делается внутри горутин
- Нет проверки ошибок

Дополнительные вопросы

- Шедулинг горутин, преимущества горутин (что такое горутины в Go и как они соотносятся с потоками операционной системы)
- А если URL адресов будет 500 и запустить запросы по ним в 500 горутинах, будут и все они выполняться одновременно? Как это устроено “под капотом”?
- Как изменить этот код чтобы можно было передавать любое количество URL адресов?
- Как изменить этот код чтобы максимальное время его выполнения было 2 секунды?
- Advanced: Как изменить код, если мы захотим сделать утилиту демоном, который умеет безопасно завершаться (graceful shutdown). Ожидаем рассказ про обработку сигналов.
- Expert: Как можно оптимизировать создание TCP-соединений в HTTP/1.1? (ожидаем ответ про keep-alive) Как можно управлять keep-alive в HTTP-клиенте go? (ожидаем, что скажет про пул соединений, MaxIdleConns, IdleConnTimeout) Для чего управлять пулом? (контроль нагрузки на сервис, к которому обращаемся)
- Expert: Как в Linux понять, сколько у процесса открытых соединений? Приходилось ли дебажить на таком уровне? (Ожидаем, что расскажет про `lsof`, `netstat` или `ss`)
- Expert: Какие проблемы нужно учесть при запросах к удаленным серверам? (Могут быть проблемы недоступности, приемлемым решением было бы использование паттерна Circuit breaker).

Неблокирующий вызов функции (medium)

Условие задачи

- Есть функция `unpredictableFunc`, работающая неопределённо долго и возвращающая число. Её тело нельзя изменять (представим, что внутри сетевой запрос).
- Нужно написать обёртку `predictableFunc`, которая будет работать с заданным фиксированным таймаутом (например, 1 секунду).

```
package main
```

```
import (  
    "fmt"  
    "math/rand"
```

```

    "time"
)

func init() {
    rand.Seed(time.Now().UnixNano())
}

// Есть функция, работающая неопределённо долго и возвращающая число.
// Её тело нельзя изменять (представим, что внутри сетевой запрос).
func unpredictableFunc() int64 {
    rnd := rand.Int63n(5000)
    time.Sleep(time.Duration(rnd) * time.Millisecond)

    return rnd
}

// Нужно изменить функцию обёртку, которая будет работать с заданным таймаутом
// (например, 1 секунду).
// Если "длинная" функция отработала за это время - отлично, возвращаем результат.
// Если нет - возвращаем ошибку. Результат работы в этом случае нам не важен.
//
// Дополнительно нужно измерить, сколько выполнялась эта функция (просто вывести в
// лог).
// Сигнатуру функцию обёртки менять можно.
func predictableFunc() int64 {
    return unpredictableFunc()
}

func main() {
    fmt.Println("started")

    fmt.Println(predictableFunc())
}

```

Что проверяем

- Базовое знание горутин
- Применение каналов для передачи результата из горутин
- Понимание работы context
- Работа с select для мультиплексирования каналов
- Работа с временем выполнения операций (прообраз работы с метриками)

Вариант решения


```
const defaultTimeout = 1 * time.Second
```

```
// Нужно написать обёртку, которая будет работать с заданным таймаутом (например, 1 секунду).
```

```
// Если "длинная" функция отработала за это время - отлично, возвращаем результат.
```

```
// Если нет - возвращаем ошибку. Результат работы в этом случае нам не важен.
```

```
//
```

```
// Дополнительно нужно измерить, сколько выполнялась эта функция (просто вывести в лог).
```

```
// Сигнатуру обёртки менять можно.
```

```
func predictableFunc(ctx context.Context) (int64, error) {
```

```
    now := time.Now()
```

```
    defer func(){
```

```
        fmt.Println("elapsed:", time.Since(now))
```

```
    }()
```

```
    var cancel context.CancelFunc
```

```
    if _, hasDeadline := ctx.Deadline(); !hasDeadline {
```

```
        ctx, cancel = context.WithTimeout(ctx, defaultTimeout)
```

```
        defer cancel()
```

```
    }
```

```
    resCh := make(chan int64, 1)
```

```
    go func() {
```

```
        i := unpredictableFunc()
```

```
        resCh <- i
```

```
        close(resCh)
```

```
    }()
```

```
    select {
```

```
        case res := <-resCh:
```

```
            return res, nil
```

```
        case <-ctx.Done():
```

```
            // timeout
```

```
            return 0, ctx.Err()
```

```
    }
```

```
}
```

```
func main() {
```

```
    fmt.Println("started")
```

```
    res, err := predictableFunc(context.Background())
```

```
    if err != nil {
```

```
        fmt.Println(err)
```

```
}  
fmt.Println(res)  
}
```

Возможные вариации решения

- кандидат может сказать, что было бы здорово заменить сигнатуру `unpredictableFunc`, чтобы она принимала `context.Context` и работала с ним "нативно". Отмечаем плюсики, отвечаем "это хороший вариант, но давай представим, что это по какой-то причине невозможно. Как бы ты это обошёл?"
- нормальный вариант, если кандидат просто принимает контекст на вход функции, а таймаут задаёт снаружи в `main()` (то есть нет дефолтного таймаута, как в примере решения)
- можно вместо `ctx` принимать на вход таймаут и работать с `<-time.After(t)` - главное, чтобы была асинхронная обработка с каналом. В этом случае нужно спросить, знает ли про `Context`.

Возможные ошибки

- При создании контекста забыл обработать `context.CancelFunc` - стоит уточнить, зачем она нужна (ответ - чтобы корректно завершить работу таймера внутри контекста и освободить ресурсы)
- При логгировании времени сделал что-то типа `defer fmt.Println("elapsed:", time.Since(time.Now()))` - нужно будет спросить про работу `defer` и разрешение параметров вызова
- Создан небуферизированный канал - и горутина навсегда заблокировалась на запись в случае таймаута
- Нет проверки ошибок

Дополнительные вопросы

- Как работает `select`, если в нескольких каналах одновременно есть данные (ответ - недетерминировано, на порядок обработки нельзя закладываться)

[Теоретические вопросы к секции "Backend платформа Go" \(medium\)](#)

Эти вопросы стоит задавать, если по каким-то причинам не получается их охватить практическими задачами.

Вопрос №1 Что такое горутины в Go и как они соотносятся с потоками операционной системы?

Basic

- Легковесность
- Переключение происходит на уровне приложения (user space)
- Знает про GOMAXPROCS
- Знает про M:N модель

Advanced

- Знает точки переключения контекста
- Знает о том, что горутина, исполняющая блокирующий системный вызов, оккупирует отдельный поток ОС
- Знает размер стека новой горутины (хотя бы порядок)
- Знает про дорогостоящие переключения регистров при шедулинге потоков, кол-во памяти на поток операционной системы (порядок)
- Понимает как найти и предотвратить утечки горутин

Expert

- Знает про шедулинг горутин рантаймом - про привязку к P, про глобальную очередь горутин

Вопрос №2 Чем занимается runtime в Go?

- Менеджмент потоков
- Шедулинг горутин
- Менеджмент памяти
- Сборка мусора

Вопрос №3 За счет чего Go позволяет работать с большим количеством сетевых соединений одновременно?

Basic

- Netpoll, неблокирующие сокеты

Advanced

- Epoll, kqueue, IOCP

- Знает преимущества epoll над select, poll
- Знает какие события в дескрипторе можно зарегистрировать

Expert

- Работал с epoll, kqueue писал свою реализацию
- Знает отличия edge-triggered от level-triggered

Вопрос №4 Что такое каналы?

Basic

- Прimitives синхронизации между горутинами
- Буферизированные и небуферизированные
- Знает как работает select
- Знает как они устроены

Advanced

- Преимущества/недостатки по сравнению с локами
- Знает что запись в закрытый канал паникует, а в nil channel заблокируется
- CSP

Expert

- ?

Вопрос №5 Расскажите о примитивах синхронизации и зачем они нужны?

- Каналы, локи, атомики, sync.WaitGroup, sync.Cond, sync.Once ...
- Проблемы: deadlock, race condition
- Отличия Mutex и RWMutex

Вопрос №6 Что такое интерфейс?

Basic

- Набор методов, которые должен иметь тип, удовлетворяющий интерфейсу
- Пустой интерфейс

Advanced

- Знает что из себя представляет интерфейс внутри: (value, type)

Expert

- nil-интерфейс

Вопрос №7 Использование памяти

Basic

- Чем отличаются емкость (cap) от размера (len) в slice и map?
- Как происходит выделение памяти, если в slice не хватает места?

Advanced

- Как происходит сборка мусора?

Вопрос №8 Написание тестов

Basic

- Какие виды тестирования бывают?
- Какие виды тестирования поддерживает команда go test "из коробки"?
- Что такое моки и как их использовать?
- Какие полезные аргументы поддерживает команда go test?

Вопрос №9 Профилирование

Basic

- Сталкивался ли с профилирование Go-программ? Какие инструменты для этого существуют? Как применяются?
- Знает о доступных тулзах, pprof, trace.
- Знает когда нужно профилировать и в общем виде какую информацию можно подчерпнуть из профилей

Advanced

- Профилировал, помнит какие именно вещи можно посмотреть (cpu, mutex, heap inuse_space, alloc_space, inuse_objects, alloc_objects)

Expert

- Знает как работают профилировщики, влияние включения профилировщика на производительность программы

